

# P-Machine VM Specification v1

## 1. Overview

This virtual machine is a modular stack-oriented runtime designed for:

- ARM32-oriented execution
- unified `byte[]` memory model
- simple and efficient bytecode execution
- modular loading/linking
- ownership-aware object lifetime
- simple mark & sweep garbage collector

The VM uses:

- little-endian binary format
- 32-bit addresses and offsets (i32)
- stack frames aligned to 4 bytes
- object layout aligned to 4 bytes
- position-independent module code using CS-relative addressing

---

## 2. Memory Model

The VM memory model is based on a single unified byte array.

byte[] memory

Runtime layout:

+-----+  
| Module Header |  
+-----+  
| Constant Pool |  
+-----+  
| Static Globals |  
+-----+  
| Code Section |  
+-----+  
| Stack |  
| grows downward |  
+-----+  
| Heap |  
| grows upward |  
+-----+

All addresses and offsets are i32.

---

## 3. Alignment Rules

All runtime structures are aligned to 4 bytes.

Includes:

- objects
- arrays
- strings
- stack frames
- globals
- locals

Alignment helper:

$$\text{align4}(x) = (x + 3) \& \sim 3$$

Small integer types:

| Type | Stored Size | Runtime Arithmetic |
|------|-------------|--------------------|
| i8   | 1 byte      | promoted to i32    |
| i16  | 2 bytes     | promoted to i32    |
| i32  | 4 bytes     | native             |
| i64  | 8 bytes     | native             |

Floating point:

| Type | Size    |
|------|---------|
| f32  | 4 bytes |
| f64  | 8 bytes |

References and owner references are 32-bit pointers.

---

## 4. Module Model

Each source module generates one binary module file.

A module may contain:

- constants
- globals
- properties
- functions
- classes
- imports

All globals are private to the module.

Constants and functions may be public or private.

Classes may be public or private.

Properties may be public or private.

---

## 5. Module Loading

When a module is loaded:

1. binary is loaded into memory
2. globals space is allocated
3. imports are recursively loaded
4. imports are resolved
5. runtime metadata is created
6. module initializers are executed

The first loaded module is the entry module.

After all modules are initialized:

```
main(String[] args)
```

from the first module is executed.

Other module main() functions are ignored.

---

## 6. Code Segment Model

Each module owns its own code segment base:

```
CS = Code Segment base
```

All internal code and global references are relative to CS.

Example:

```
LEA CS + offset
```

This eliminates internal relocation.

---

## 7. Function Calls

Function calls are divided into:

| Call Type    | Description             |
|--------------|-------------------------|
| CALL_LOCAL   | call inside same module |
| CALL_IMPORT  | call to another module  |
| CALL_VIRTUAL | virtual method dispatch |
| CALL_NATIVE  | runtime/native call     |

CALL\_LOCAL does not modify CS.

CALL\_IMPORT changes CS to target module CS.

---

## 8. Calling Convention

Stack grows downward.

Stack frame layout:

```
| return value slot | optional
| parameters       |
| return PC        |
| previous CS      |
| old BP           |
| locals           |
| temporaries      |
```

Rules:

- caller reserves return slot
- caller pushes parameters
- CALL stores PC and previous CS

- callee allocates locals
- RET restores PC, CS and BP

Methods:

- instance methods receive implicit this
- static methods do not receive this

The implicit this is parameter 0.

---

## 9. Ownership Model

References may be:

- normal references
- owner references

Syntax:

```
var [owner] name: ClassType
```

Owner references transfer ownership.

Assigning owner to owner:

- transfers pointer
- source becomes null

Owner destruction:

- destroys owned object
- recursively destroys owned fields

The compiler automatically inserts cleanup blocks.

Conceptual model:

```
try {  
    ...  
} finally {  
    destroy(owner variables)  
}
```

The VM itself is ownership-unaware.

Ownership logic is generated by the compiler.

---

## 10. Garbage Collector

GC type:

```
Mark & Sweep
non-moving
non-generational
```

The GC acts as fallback memory recovery.

Ownership provides deterministic destruction.

### Heap Objects

All objects reside in heap memory.

Objects are never moved.

### GC Roots

Roots include:

- stack references
- globals
- static references
- owner references

### Allocation

Allocator uses:

```
first-fit free list
```

GC is triggered on allocation failure.

---

## 11. Object Layout

All objects use the following header:

```
offset 0   classId   u32
offset 4   flags     u32
offset 8   size      u32
```

Fields begin at offset 12.

Example:

```
class Point
{
    i32 x
    i32 y
}
```

Memory layout:

```
classId
flags
size
x
y
```

---

## 12. Inheritance Layout

Inheritance is flat.

Parent fields are stored before child fields.

Example:

```
class A
{
    i32 a
}

class B : A
{
    i32 b
}
```

Layout of B:

```
header
a
b
```

Field offsets remain stable in subclasses.

---

## 13. Arrays

Arrays may be:

- fixed size local/global arrays
- dynamic heap arrays

Heap array layout:

```
classId  
flags  
size  
length  
elements...  
padding
```

All arrays are aligned to 4 bytes.

Embedded fixed arrays inside objects are padded to multiples of 4 bytes.

---

## 14. Strings

Strings are immutable objects.

String layout:

```
classId  
flags  
size  
length  
hash  
data...  
padding
```

Strings are heap objects.

---

## 15. Runtime Metadata

### ClassInfo

```
classId
objectSize
fieldOffsets[]
referenceBitmap
vtable
parentClass
```

### FunctionInfo

```
address
localsSize
argsSize
flags
```

---

## 16. Module Binary Format

Module binary layout:

```
+-----+
| Header          |
+-----+
| Constant Pool   |
+-----+
| Import Table    |
+-----+
| Export Metadata |
+-----+
| Globals Metadata|
+-----+
| Class Metadata  |
+-----+
| Function Metadata|
+-----+
| Code Section    |
+-----+
```

All offsets are i32.

All values are little-endian.

---

## 17. Header Format

Suggested header fields:

```
magic
vmVersion
moduleVersion
flags

offsetConstantPool
offsetImports
offsetExports
offsetGlobals
offsetClasses
offsetFunctions
offsetCode
```

---

## 18. Imports and Exports

Imports:

```
moduleName
symbolName
symbolType
```

Runtime exports:

- public functions
- public classes

Compile-time only exports:

- public constants

Constants from other modules are resolved by the compiler.

The VM does not import constants at runtime.

---

## 19. Constructors

Constructors are normal functions with implicit this.

Object creation:

```
ALLOC  
CALL constructor
```

The VM does not contain a dedicated NEW\_OBJECT instruction.

---

## 20. VM Philosophy

The VM is intentionally simple:

- simple bytecode
- compiler-driven ownership
- no moving GC
- no runtime verifier
- no complex relocation
- no mandatory exceptions system

Complexity belongs primarily in the compiler.

The VM remains:

- deterministic
- modular
- efficient
- ARM32-friendly
- easy to debug